

Voice Agent Platform

Local start & deployment guide — backend (agent), frontend (console), and the test results behind this guide. Generated from the actual project source; commands below are not guesses.

1. Architecture at a glance

Two independently-run pieces, plus infrastructure they both depend on:

- **Token service** (FastAPI, `app/security/token_service.py`) — issues LiveKit access tokens. Two endpoints: `POST /v1/dev/token` (dev-only bootstrap) and `POST /v1/livekit/token` (production token exchange, requires a bearer JWT).
 - **Agent worker** (`app/agent_entrpoint.py`) — a LiveKit Agents worker process. It joins rooms, runs STT (Sarvam) → LLM (Google) → TTS (Sarvam), persists sessions/turns/tool calls to Postgres, and is started via the LiveKit Agents CLI, not as an HTTP server.
 - **Frontend console** (React + TypeScript + Vite + `livekit-client`) — the UI covered in the earlier build: connects to LiveKit, publishes the mic, and renders transcript/agent-state/tool events from real room events.
 - **Infrastructure** — PostgreSQL (session/turn/audit/tool-call persistence), a LiveKit server (Cloud or self-hosted), and provider credentials for Sarvam (STT/TTS), Google (LLM), and Moss/Qdrant (context retrieval).
-

2. Prerequisites

Component	Version used in this pass	Notes
Python	3.12.3 (Linux)	The project's committed agent / <code>.venv</code> was built on Windows (has <code>Scripts/</code> , not <code>bin/</code>) and is not usable on Linux/macOS — create a fresh <code>venv</code> there instead.
Node.js	22.22.2	Vite 8 + React 19; Node 20+ recommended.
PostgreSQL	16.15	Required for every DB-backed backend test and for the token service / agent worker at runtime.
LiveKit server	not exercised in this pass	Cloud project or self-hosted; needed for any real voice session, dev or prod.

3. Backend — local start

3.1 Create a Linux-native virtual environment

```
cd voice-agent-platform/agent
python3 -m venv .venv-linux
source .venv-linux/bin/activate
pip install -r requirements.txt
pip install -r requirements-dev.txt # pytest, mypy, etc.
```

This pulls in `sentence-transformers` and related ML packages used by `app/context/`. They are not needed to run the pure-logic or DB-backed test suites (see §5), only to actually run the agent worker end to end.

3.2 Required environment variables

Variable	Where	Notes
DATABASE_URL	both	e.g. postgresql+asyncpg://user:pass@host:5432/vap. Read directly by app/db/base.py and Alembic.
JWT_SECRET	both	HS256 signing secret for the application JWT (token exchange trust boundary).
JWT_ISSUER	token service	Must match the iss claim on tokens presented to /v1/livekit/token.
LIVEKIT_API_KEY / LIVEKIT_API_SECRET	both	From your LiveKit project. Used to mint LiveKit access tokens server-side — never expose these to the frontend.
SARVAM_API_KEY	agent worker	Sarvam STT/TTS.
MOSS_PROJECT_KEY	agent worker	Moss context provider.
ENVIRONMENT	token service	Set to development to enable /v1/dev/token; any other value (or unset) 404s it.

None of the variables above belong in the frontend's .env. The frontend only ever needs VITE_API_BASE_URL and VITE_LIVEKIT_URL — see §4.

3.3 Database migrations

```
export DATABASE_URL=postgresql+asyncpg://user:pass@localhost:5432/vap
alembic upgrade head
```

3.4 Run the two processes

The token service (FastAPI/uvicorn) and the agent worker are separate long-running processes:

```
# Terminal 1 – token service, http://localhost:8000
uvicorn app.security.token_service:app --reload --port 8000

# Terminal 2 – LiveKit agent worker (connects out to your LiveKit server)
python -m app.agent_entrypoint dev
```

dev is the LiveKit Agents CLI's local-development subcommand (agents.cli.run_app at the bottom of agent_entrypoint.py); use start for a production-style run.

4. Frontend — local start

```
cd voice-agent-platform/frontend
cp .env.example .env # set VITE_LIVEKIT_URL to your LiveKit server
npm install
npm run dev          # http://localhost:5173
```

VITE_API_BASE_URL defaults to http://localhost:8000 in .env.example, matching the token service in §3.4.

Production build

```
npm run build    # tsc -b && vite build → dist/  
npm run preview # sanity-check the built output locally
```

5. Test results (this pass)

All numbers below are from actually running the suites in a Linux sandbox, not estimated.

5.1 Backend — pytest

The shipped agent/.venv could not be used (Windows-only). A fresh Linux venv was created and requirements.txt + requirements-dev.txt installed into it.

Run	Result	Notes
Without a live Postgres	20 passed, 66 errored	Errors are KeyError: 'DATABASE_URL' at fixture setup — expected, not a code defect. The 20 passing tests are pure logic (JWT encode/decode, Sarvam STT/TTS with mocked HTTP, tool-contract classification, etc.).
With a local Postgres 16 instance	146 passed, 0 failed	Full suite, all 15 test files: identity/auth, session boundary, tenant isolation, tool persistence, session manager, create-ticket tool, Sarvam STT/TTS, embeddings, Moss provider, Qdrant provider, stub tool definitions, the non-live-session parts of agent_entrypoint.py, and CORS configuration (new — see §6.1).

Line coverage (--cov=app, with Postgres)

Coverage improved on this pass: 5 modules that previously had 0% coverage — agent_entrypoint.py, context/embeddings.py, context/moss_provider.py, context/qdrant_provider.py, and tools/definitions.py — now have dedicated unit tests. Overall project coverage moved from 66% to 83%.

Module	Coverage	Notes
context/embeddings.py	100%	New. sentence-transformers is stubbed out entirely (module import is lazy) — tests cover the query-prefix convention and the executor-based async wrapping, not the real model.
context/moss_provider.py	100%	New. Real Moss SDK client patched out — tests cover tenant-scoped index naming, index-load caching, and the create/add-docs fallback.
context/qdrant_provider.py	100%	New. Mocked AsyncQdrantClient — tests cover collection creation, tenant/knowledge-type filtering, and result mapping.
tools/definitions.py	99%	New. Confirms every stub tool validates input and reports an honest failure, never a fabricated success.
agent_entrypoint.py	60%	New, partial. FieldOpsAssistant's two function-tool methods and _record_boundary_failure are covered. entrypoint() itself (lines 146-255) still needs a live JobContext/room and is not exercised — consistent with the module's own docstring.

Module	Coverage	Notes
Security / identity / JWT / session boundary	84 – 100%	Unchanged — already covered by the existing DB-backed test files.
Tools (create_ticket, executor, registry, idempotency)	87 – 98%	Unchanged.
Sarvam STT / TTS	67 – 76%	Unchanged — core paths covered, some streaming edge-cases aren't.
context/orchestrator.py	31%	Not addressed this pass — retrieval-trigger branching and the concurrent moss/qdrant/live-api gather path are still largely untested.
context/live_api.py	0%	Dead code, not a coverage gap: this file defines a LiveOperationalAPI class that nothing imports. agent_entrypoint.py uses an identical class defined directly inside orchestrator.py instead. Worth deleting or consolidating, not worth testing as-is.

5.2 Frontend — build / lint

Command	Result
npm run lint (eslint .)	Clean — 0 errors, 0 warnings
npm run build (tsc -b && vite build)	Clean — 0 type errors; production bundle emitted to dist/
Automated tests (vitest/jest)	None configured in package.json — nothing to run

The frontend has no automated test suite (no test script, no vitest/jest config). Type-checking (tsc) and linting (eslint) are the only automated checks currently in place.

6. Deployment — frontend on Vercel, backend on Render

This section reflects what was actually verified in this session: a real `.env` was checked line-by-line against every environment variable the code reads, two real bugs were found and one was fixed (CORS — see §6.1). Render's and Vercel's exact field names below were confirmed against their current documentation via web search, not recalled from memory. What could **not** be verified: this sandbox has no network path to Render, Vercel, LiveKit Cloud, Neon, Sarvam, or Moss (outbound traffic is allowlisted to package registries only) — so nothing below was clicked through on the actual platforms, and the real external services were never actually reached.

6.1 Fixed this pass: CORS

The token service had no CORS configuration at all — confirmed live with a real `OPTIONS` preflight against a running instance, which came back `405 Method Not Allowed` with no `Access-Control-Allow-*` headers. Since a Vercel frontend and a Render backend are always different origins, every browser `fetch()` call would have been blocked client-side regardless of whether the request itself succeeded — invisible to `curl/httpx`-based tests, which is exactly why nothing caught it earlier. Added `CORSMiddleware` gated by a new `ALLOWED_ORIGINS` env var (comma-separated, closed by default — no wildcard). Re-verified live: a preflight from an allowed origin now returns the correct `Access-Control-Allow-Origin`; an unlisted origin still gets nothing. Covered by 4 new tests in `tests/test_cors.py`.

6.2 What the uploaded `.env` is missing or wrong

Variable	Issue found	Fix
GOOGLE_API_KEY	Not present. Confirmed by reading <code>livekit/plugins/google/llm.py</code> : with no <code>api_key</code> argument (<code>agent_entrypoint.py</code> doesn't pass one), it falls back to <code>os.environ.get('GOOGLE_API_KEY')</code> and raises if unset.	Add a Gemini API key from Google AI Studio — a separate credential from everything else in the file.
DATABASE_URL	Scheme is plain <code>postgresql://</code> with <code>sslmode=require&channel;_binding=require</code> . Verified in this session: <code>create_async_engine</code> with that exact string raises <code>TypeError ('unexpected keyword argument sslmode')</code> the moment a connection is attempted — <code>asyncpg</code> doesn't accept <code>libpq</code> -style <code>ssl</code> params.	Change scheme to <code>postgresql+asyncpg://</code> and replace <code>sslmode=require&channel;_binding=require</code> with <code>ssl=require</code> . Verified in this session: the corrected URL parses and reaches the network layer cleanly (network itself is blocked in this sandbox, but the URL is no longer the problem).

Everything else in the uploaded `.env` matched what the code actually reads, one-to-one — `DATABASE_URL`/`JWT_SECRET`/`JWT_ISSUER`/`ENVIRONMENT`, `LIVEKIT_URL`/`LIVEKIT_API_KEY`/`LIVEKIT_API_SECRET`, `MOSS_PROJECT_ID`/`MOSS_PROJECT_KEY`, `SARVAM_API_KEY`, `QDRANT_LOCATION`, `GEMINI_MODEL`. No unused or stray variables either.

6.3 Backend on Render

A Blueprint is included at `agent/render.yaml` defining two services — a web service for the token service and a background worker for the agent. Field names (`type/runtime/rootDir/buildCommand/startCommand/healthCheckPath/envVars/sync`) were checked against Render's current Blueprint YAML Reference.

- Render dashboard → New → Blueprint → point at the repo containing this project. If `render.yaml` sits at `agent/render.yaml` rather than the repo root, Render needs it at the repo root — move or symlink it, or set the Blueprint's root accordingly.
- Every secret in `render.yaml` is `sync: false` deliberately — Render prompts for the value in its dashboard instead of it living in a committed file. Paste the corrected `DATABASE_URL` (§6.2) and the rest of the real values there, not into `render.yaml`.
- The worker service is set to Render's standard plan, not starter — sentence-transformers pulls in torch, and starter's RAM is tight for that plus a live LiveKit session.
- Deploy the web service first, note its `onrender.com` URL — that's `VITE_API_BASE_URL` for the Vercel side (§6.4). `ALLOWED_ORIGINS` on Render needs the Vercel URL in the other direction, so expect to circle back and set it after §6.4 gives you that URL.

6.4 Frontend on Vercel

Vercel auto-detects a Vite project with no config file needed (confirmed against Vercel's current Vite docs) — no `vercel.json` was added; this app also has no client-side routing, so no SPA-fallback rewrite is needed either.

- New Project → import the repo. Since this is a monorepo (`agent/` + `frontend/` side by side), set **Root Directory** to `frontend` — the one setting Vercel won't infer on its own. Framework Preset, Build Command, and Output Directory can all stay on their Vite defaults.
- Project Settings → Environment Variables: `VITE_API_BASE_URL` = the Render web service URL from §6.3, `VITE_LIVEKIT_URL` = your LiveKit `wss://...` URL. Vite inlines both at build time, so redeploy after setting or changing either.
- Once the Vercel URL exists, go back to Render and set `ALLOWED_ORIGINS` to it (plus `http://localhost:5173` if you also want local dev to reach the deployed backend) — this closes the loop from §6.3.

6.5 Before making the Render URL public

ENVIRONMENT=development leaves POST /v1/dev/token live with no authentication. Anyone who has (or finds) your Render URL can mint a valid app JWT and LiveKit token for any `tenant_slug/external_id` they type in — no login, no check. This is fine for your own testing but is a real open door the moment the URL is reachable. This project's own code deliberately does not implement production token issuance — per the docstring in `app/security/token_service.py`, that's an explicitly out-of-scope gap, not a flag to flip. There is no safe production configuration of this endpoint as it stands; treat the deployed URL as effectively an open demo, not a real login boundary, until a real auth flow sits in front of it.

6.6 Known limitations of this deployment shape

- `QDRANT_LOCATION=:memory:` means every upserted knowledge document lives only in that one worker process's RAM — gone on every restart/redeploy, and not shared if you ever run more than one worker instance. Fine for a single always-on demo worker; swap for a real Qdrant Cloud/self-hosted URL otherwise — `app/context/qdrant_provider.py` only needs an `AsyncQdrantClient`, it doesn't care how it was constructed.
 - No Dockerfile or CI pipeline exists in the repo — `render.yaml` uses Render's native Python runtime (pip install), which is enough to deploy but doesn't give you a repeatable local build parity check the way a Dockerfile would.
 - Database migrations (alembic upgrade head) are not wired into either service's build/start command — run it manually against the production `DATABASE_URL` after the first deploy and after any future schema change.
-

Generated from the actual project source and test runs in this session — not a template.